

CLIP FLIP BLIP SigLIP. . .

Introduction to CLIP Variants

Learning Transferable Visual Models From Natural Language Supervision

Radford et al., 2021, # of citations: 75060

<https://arxiv.org/abs/2103.00020>

Abstract, Introduction

- (1) SoTA methods depend on the fixed set of predetermined obj. categories
 - Learning from the raw text about images
- (2) Which caption goes to w/ which image is an efficient and scalable
- (3) Transfers to downstream tasks non-trivially

Data collection

- (1) Crawl Wikipedia for all words occurring at least 100 times + Page titles
- (2) Search for images for above queries
- (3) Balance the results by including up to 20k per a query

Contrastive Learning

- (1) Training a system to work on the proxy task of predicting which text matches which image is potentially easier
- (2) Min. the cosine sim. for $N^2 - N$ (incorrect pairing)
- (3) Max. the cosine sim. for N (correct!)

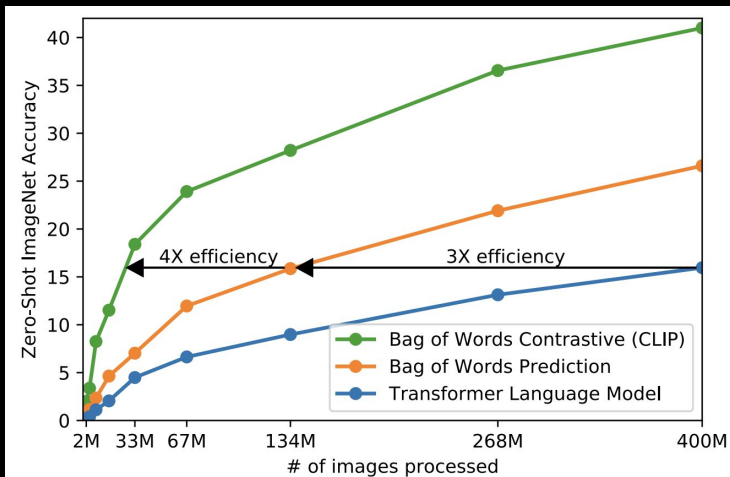
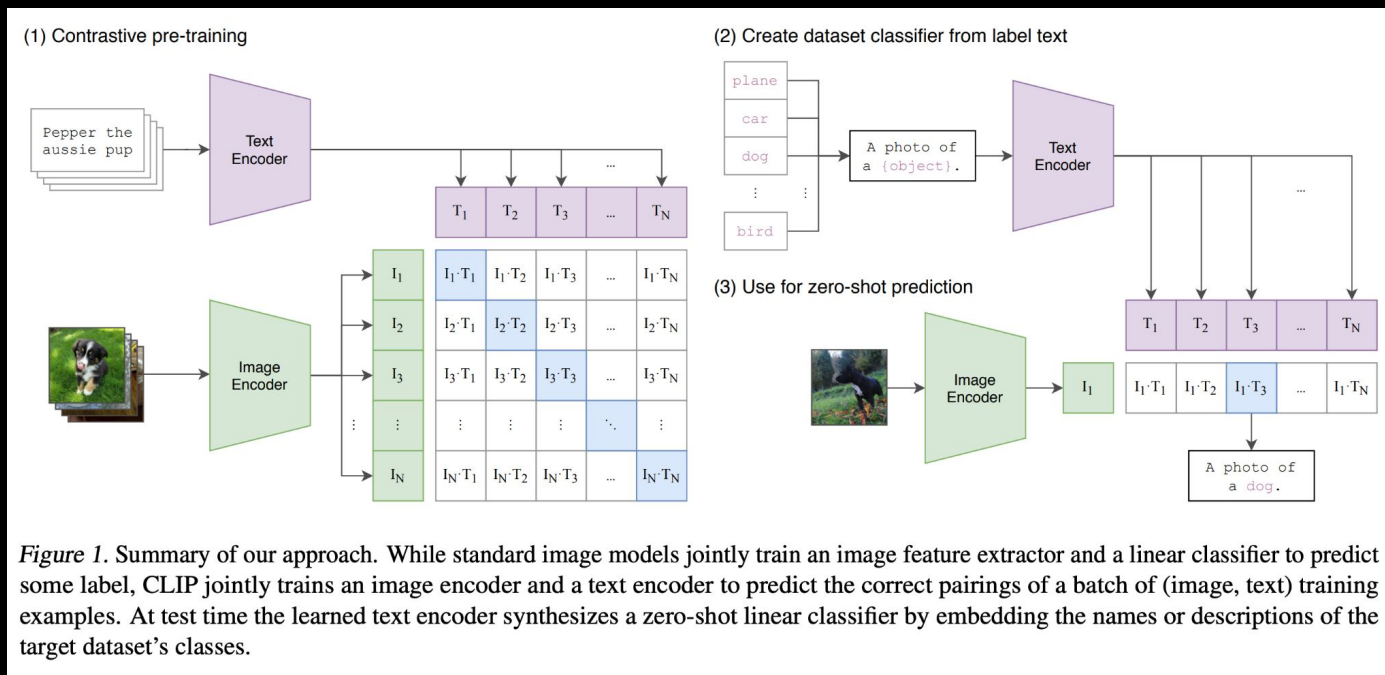


Figure 2. CLIP is much more efficient at zero-shot transfer than our image caption baseline. Although highly expressive, we found that transformer-based language models are relatively weak at zero-shot ImageNet classification. Here, we see that it learns 3x slower than a baseline which predicts a bag-of-words (BoW) encoding of the text (Joulin et al., 2016). Swapping the prediction objective for the contrastive objective of CLIP further improves efficiency another 4x.

Model Arch.



Additional methodologies and loss formulation

- (1) The only DA done is simple
rand. sq. crop
- (2) Use learnable temp. param. t
to control the range of softmax
- (3) t is optimized as a log-
parametrized multiplicative scalar

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

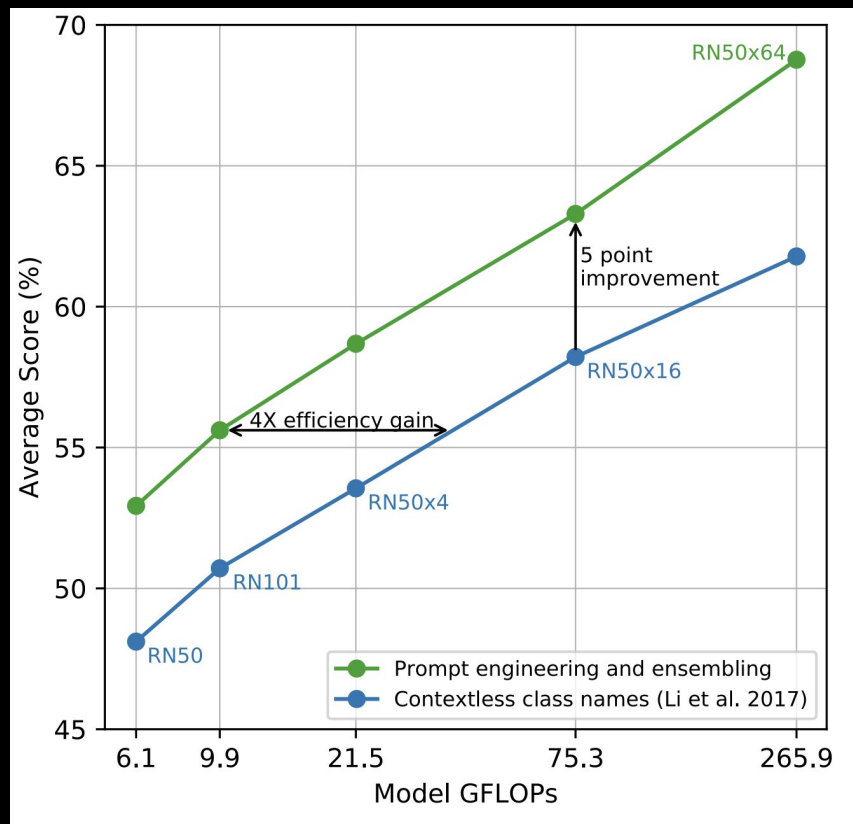
Prompt Engineering

- (1) A photo of a {}
- (2) A satellite photo of a {}

.

.

.



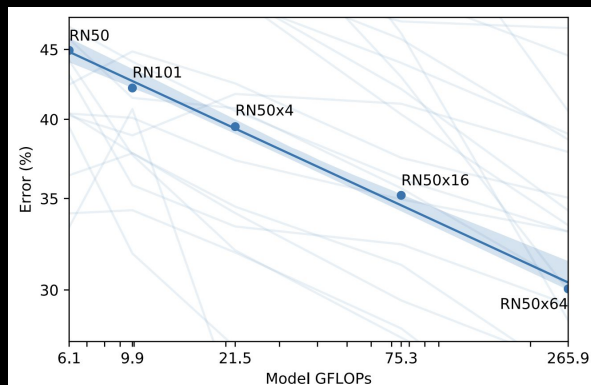
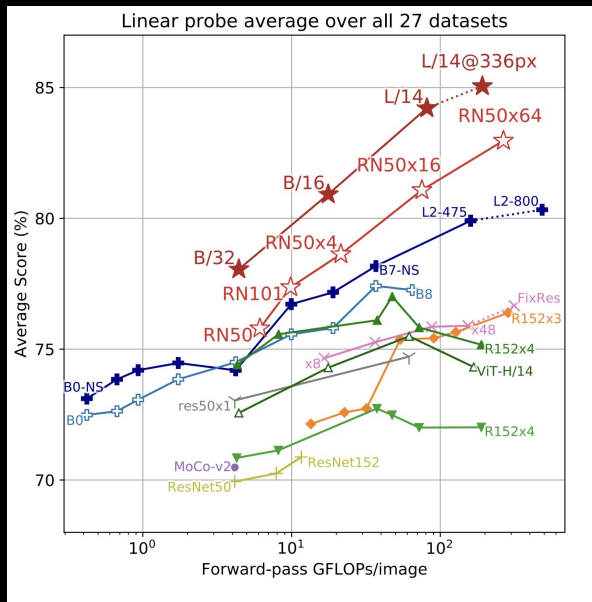
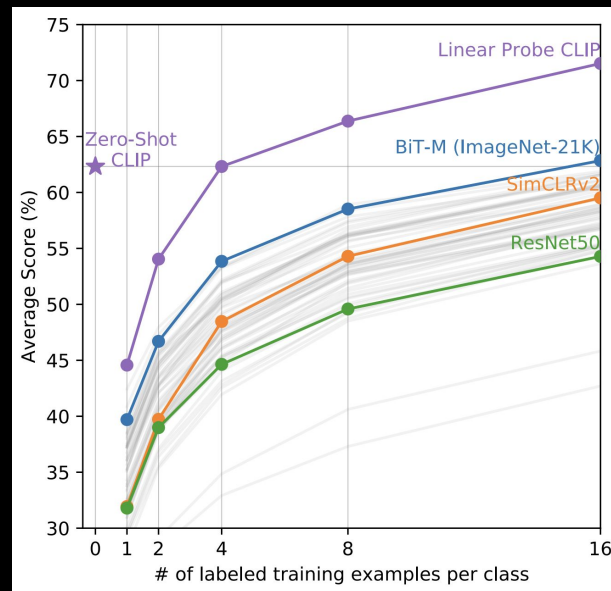


Figure 9. Zero-shot CLIP performance scales smoothly as a function of model compute. Across 39 evals on 36 different datasets, average zero-shot error is well modeled by a log-log linear trend across a 44x range of compute spanning 5 different CLIP models. Lightly shaded lines are performance on individual evals, showing that performance is much more varied despite the smooth overall trend.



Scaling law discovery, catastrophic forgetting induced by few-shot linear probing

Scaling Language-Image Pre-training via Masking

Li et al., 2022, # of citations: 517

<https://arxiv.org/abs/2212.00794>

MAE Abstract, Introduction (Crash Course)

Rationales:

- (1) ViT = Data-hungry
- (2) GPT: Autoregressive Modeling
- (3) BERT: Masked Autoencoding
- (4) Image has heavy spatial redundancy
(mask can be reversed w/ extrapolation)

<https://arxiv.org/abs/2111.06377>

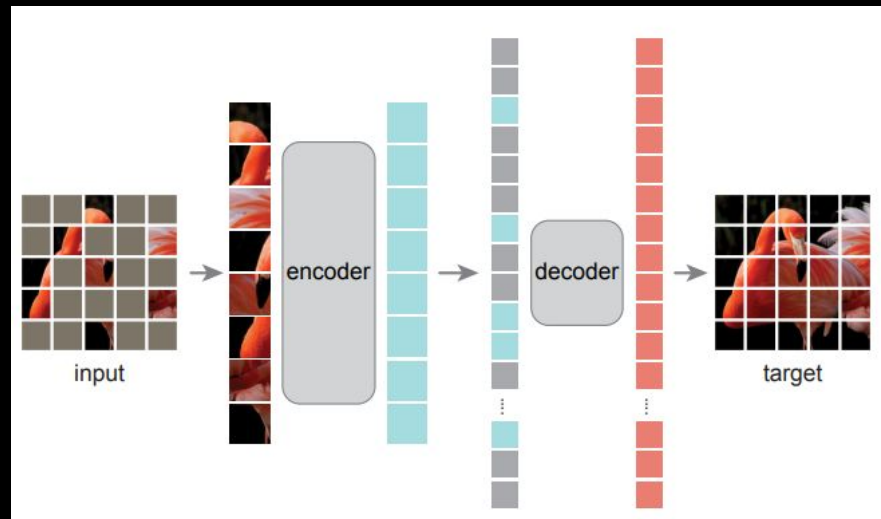
MAE Abstract, Introduction (Crash Course)

Methodologies:

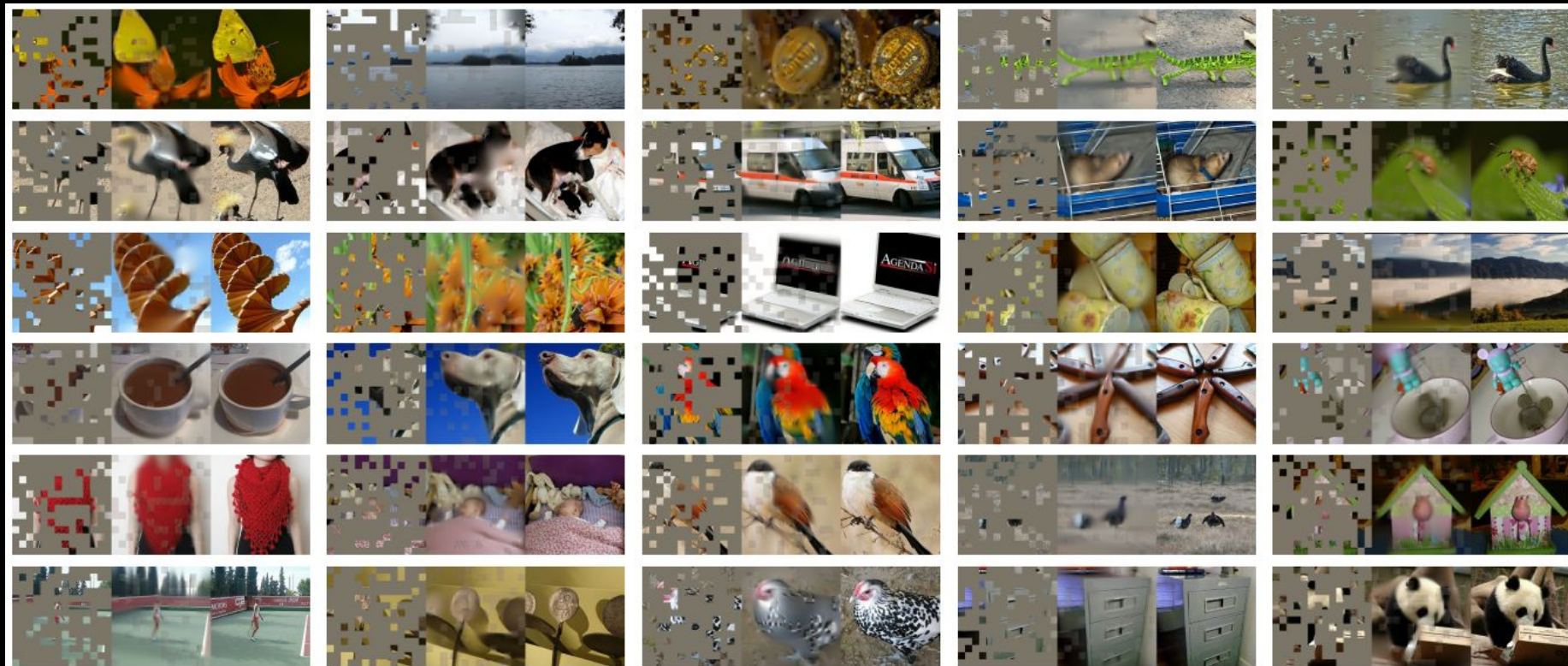
- (1) Mask image patches
→ Reconstruct
- (2) Asymmetric Encoder-Decoder
(smaller decoder)

Findings:

- (1) Masking significant portion of image
→ nontrivial performance increase
- (2) Training Acceleration (> 3x)



MAE Approach: Results



MAE Questions

Q. Use of smaller decoder?

In vision, the decoder reconstructs pixels, hence its output is of a lower semantic level than common recognition tasks; different from BERT, where the decoder predicts missing words that contain rich semantic information.

Q. Use of mask tokens in decoder input?

If the encoder uses mask tokens, it performs worse. The encoder has a large portion of mask tokens in its input in pretraining, which does not exist in uncorrupted images. By skipping the mask token in the encoder, this may also reduce training computation.

FLIP Arch.

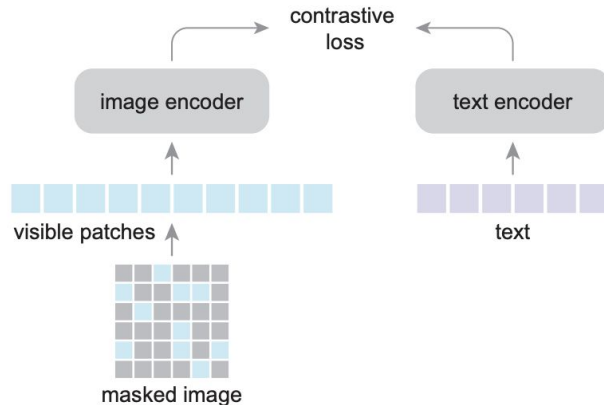


Figure 2. **Our FLIP architecture.** Following CLIP [52], we perform contrastive learning on pairs of image and text samples. We randomly mask out image patches with a high masking ratio and encode only the visible patches. We do not perform reconstruction of masked image content.

FLIP Speed-up

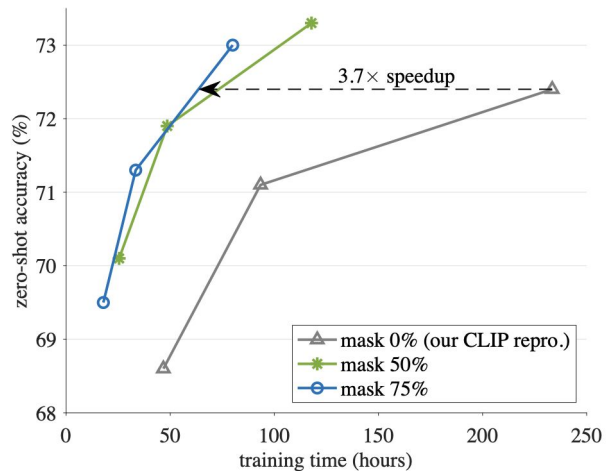


Figure 1. **Accuracy vs. training time trade-off.** With a high masking ratio of 50% or 75%, our FLIP method trains faster and is more accurate than its CLIP counterpart. All entries are benchmarked in 256 TPU-v3 cores. Training is done on LAION-400M for 6.4, 12.8, or 32 epochs, for each masking ratio. Accuracy is evaluated by zero-shot transfer on the ImageNet-1K validation set. The model is ViT-L/16 [20]. More details are in Fig. 3. As the CLIP baseline takes $\sim 2,500$ TPU-days training, a speedup of $3.7\times$ can save $\sim 1,800$ TPU-days.

FLIP Ablation study

mask	batch	FLOPs	time	acc.
0%	16k	1.00×	1.00×	68.6
50%	32k	0.52×	0.50×	69.6
75%	64k	0.28×	0.33×	68.2

(a) **Image masking** yields higher or comparable accuracy and speeds up training. Entries are subject to the same memory limit.

batch	mask 50%	mask 75%
16k	68.5	65.8
32k	69.6	67.3
64k	70.4	68.2

(b) **Batch size.** A large batch has big gains over smaller batches.

text mask	text len	time	acc.
baseline, 0%	32	1.00×	68.2
random, 50%	16	0.92×	66.0
prioritized, 50%	16	0.92×	67.8

(c) **Text masking** performs decently, but the speed gain is marginal as its encoder is smaller. Here the image masking ratio is 75%.

	mask 50%	mask 75%
w/ mask	66.4	60.9
w/ mask, ensemble	68.1	65.1
w/o mask	69.6	68.2

(d) **Inference unmasking.** Inference on intact images performs strongly even without tuning.

	mask 50%	mask 75%
baseline	69.6	68.2
+ tuning	70.1	69.5

(e) **Unmasked tuning.** The distribution shift by masking is reduced by a short tuning.

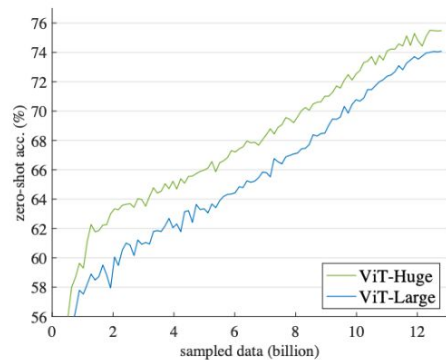
	mask 50%	mask 75%
baseline	69.6	68.2
+ MAE	69.4	67.9

(f) **Reconstruction.** Adding the MAE reconstruction loss has no gain.

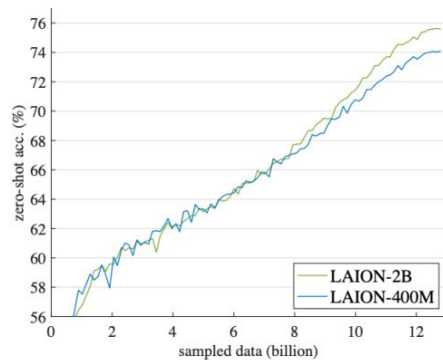
Table 1. **Zero-shot ablation experiments.** Pre-training is on LAION-400M for 6.4 epochs, evaluated by zero-shot classification accuracy on ImageNet-1K validation. The backbone is ViT-L/16 [20]. Unless specified, the baseline setting is: image masking is 50% (batch 32k) or 75% (batch 64k), text masking is 0%, and no unmasked tuning is used.

Masked pre-training requires unmasked tuning for the input distribution alignment

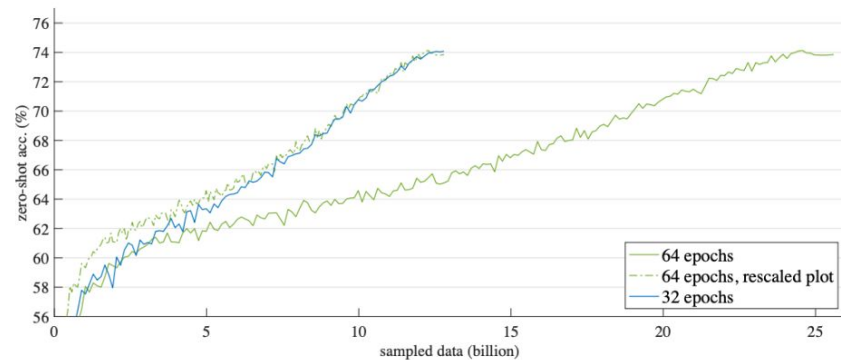
FLIP further scaling directions



(a) Model scaling



(b) Data scaling



(c) Schedule scaling

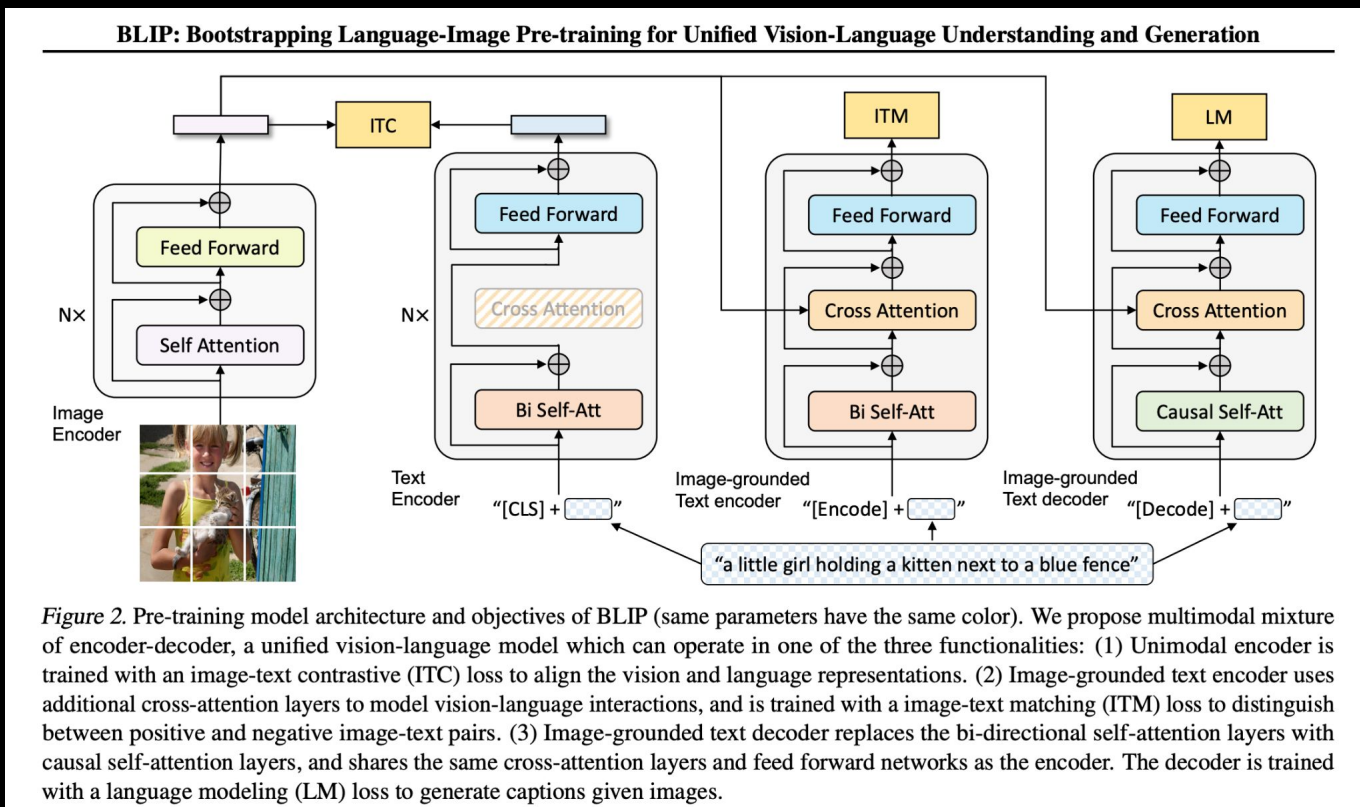
Figure 4. **Training curves of scaling.** The x-axis is the number of sampled data during training, and the y-axis is the zero-shot accuracy on IN-1K. The blue curve is the baseline setting: ViT-Large, LAION-400M data, 32 epochs (12.8B sampled data). In each subplot, we compare with scaling one factor on the baseline. In schedule scaling (Fig. 4c), we plot an extra hypothetical curve for a better visualization.

BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation

Li et al., 2022, # of citations: 10693

<https://arxiv.org/abs/2201.12086>

BLIP Arch.



BLIP CapFlt

BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation

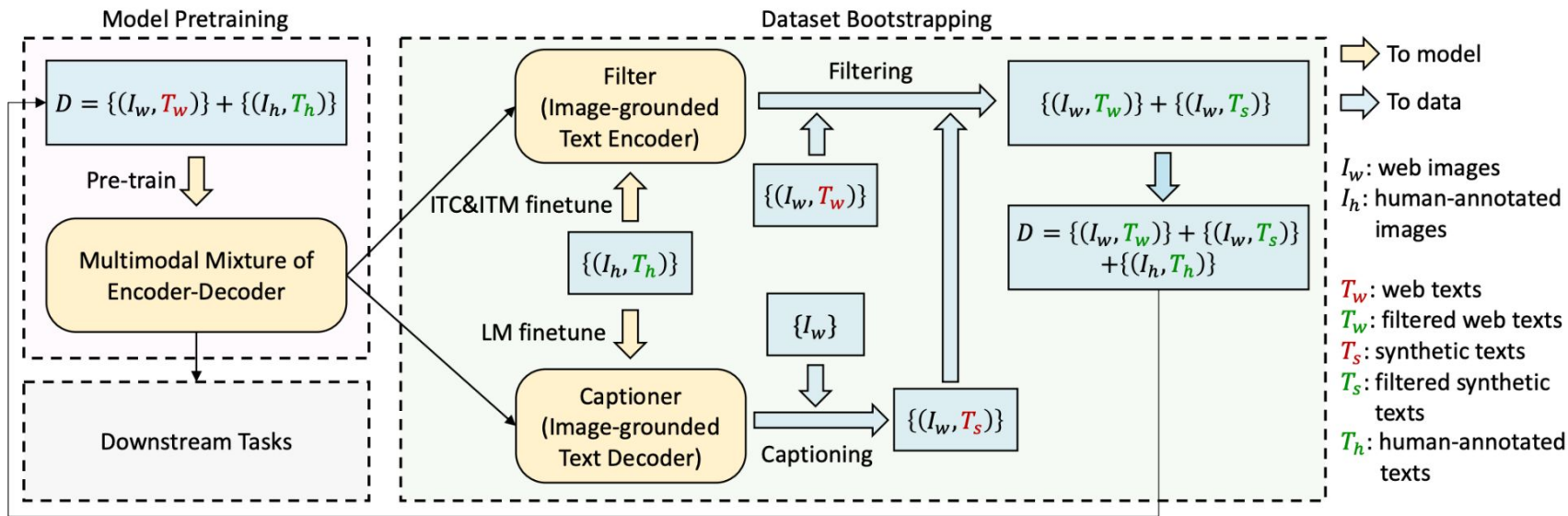


Figure 3. Learning framework of BLIP. We introduce a captioner to produce synthetic captions for web images, and a filter to remove noisy image-text pairs. The captioner and filter are initialized from the same pre-trained model and finetuned individually on a small-scale human-annotated dataset. The bootstrapped dataset is used to pre-train a new model.

BLIP Ablation study

Generation method	Noise ratio	Retrieval-FT (COCO)		Retrieval-ZS (Flickr)		Caption-FT (COCO)		Caption-ZS (NoCaps)	
		TR@1	IR@1	TR@1	IR@1	B@4	CIDEr	CIDEr	SPICE
None	N.A.	78.4	60.7	93.9	82.1	38.0	127.8	102.2	13.9
Beam	19%	79.6	61.9	94.1	83.1	38.4	128.9	103.5	14.2
Nucleus	25%	80.6	63.1	94.8	84.9	38.6	129.7	105.1	14.4

Table 2. Comparison between beam search and nucleus sampling for synthetic caption generation. Models are pre-trained on 14M images.

Layers shared	#parameters	Retrieval-FT (COCO)		Retrieval-ZS (Flickr)		Caption-FT (COCO)		Caption-ZS (NoCaps)	
		TR@1	IR@1	TR@1	IR@1	B@4	CIDEr	CIDEr	SPICE
All	224M	77.3	59.5	93.1	81.0	37.2	125.9	100.9	13.1
All except CA	252M	77.5	59.9	93.1	81.3	37.4	126.1	101.2	13.1
All except SA	252M	78.4	60.7	93.9	82.1	38.0	127.8	102.2	13.9
None	361M	78.3	60.5	93.6	81.9	37.8	127.4	101.8	13.9

Table 3. Comparison between different parameter sharing strategies for the text encoder and decoder during pre-training.

Diversity (nucleus decoding?), training/force generalization on diff. tasks using shared modules

BLIP Ablation study (continued)

Continue	Retrieval-FT (COCO)		Retrieval-ZS (Flickr)		Caption-FT (COCO)		Caption-ZS (NoCaps)	
	TR@1	IR@1	TR@1	IR@1	B@4	CIDEr	CIDEr	SPICE
Yes	80.6	63.0	94.5	84.6	38.5	129.9	104.5	14.2
No	80.6	63.1	94.8	84.9	38.6	129.7	105.1	14.4

Table 13. Continue training the pre-trained model offers less gain compared to training a new model with the bootstrapped dataset.

Re-train, do not cont. learning

Sigmoid Loss for Language Image Pre-Training

Zhai et al., 2023, # of citations: 4764

<https://arxiv.org/abs/2303.15343>

SigLip in a nutshell

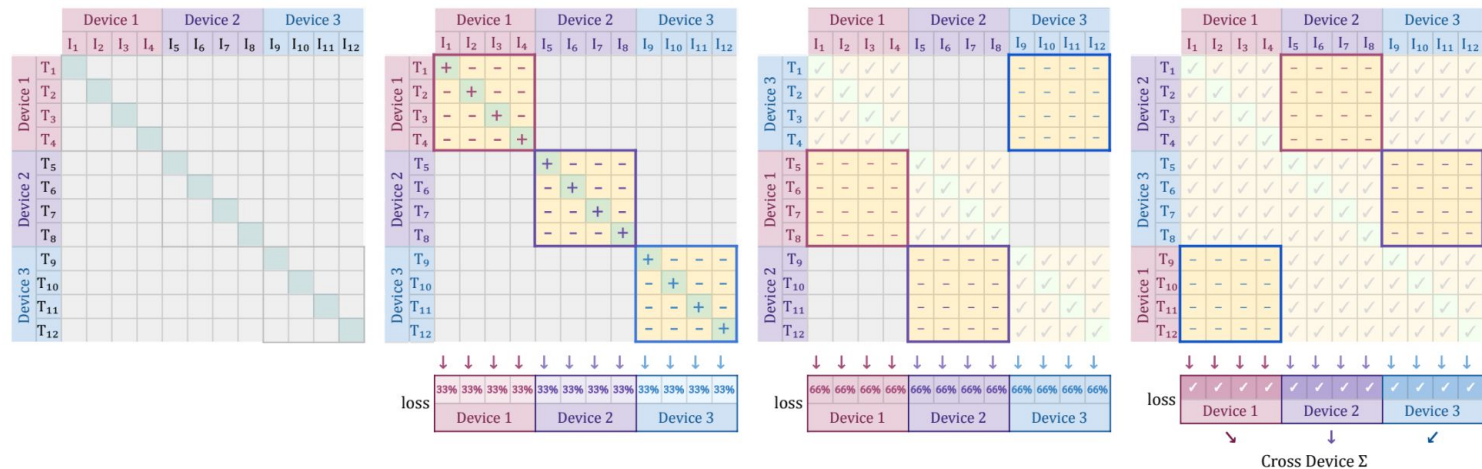


Figure 1: **Efficient loss implementation** demonstrated via a mock setup with 3 devices and a global batch size of 12. There are no all-gathers, and at any point in time only the bright yellow square (size 4×4) is materialized in memory.

SigLip in a nutshell

$$-\frac{1}{2|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \left(\overbrace{\log \frac{e^{t\mathbf{x}_i \cdot \mathbf{y}_i}}{\sum_{j=1}^{|\mathcal{B}|} e^{t\mathbf{x}_i \cdot \mathbf{y}_j}}}^{\text{image} \rightarrow \text{text softmax}} + \overbrace{\log \frac{e^{t\mathbf{x}_i \cdot \mathbf{y}_i}}{\sum_{j=1}^{|\mathcal{B}|} e^{t\mathbf{x}_j \cdot \mathbf{y}_i}}}^{\text{text} \rightarrow \text{image softmax}} \right)$$

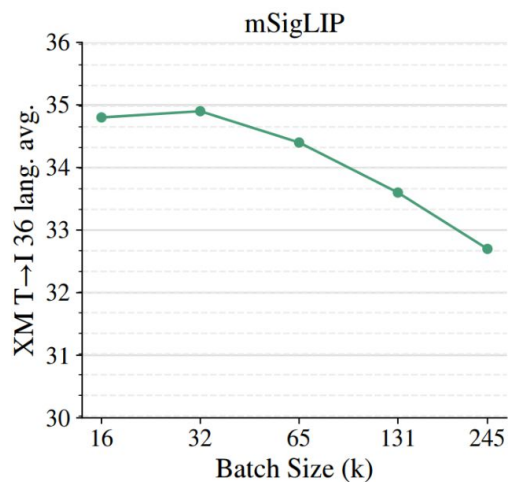
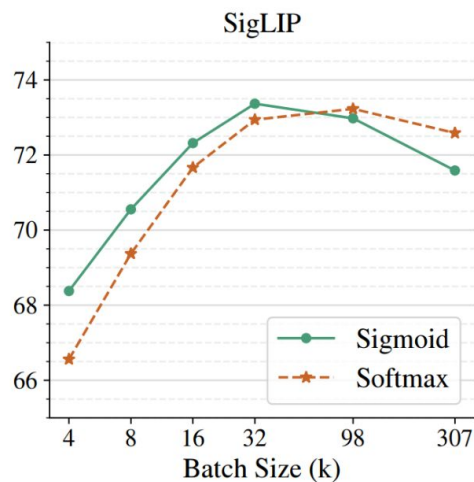
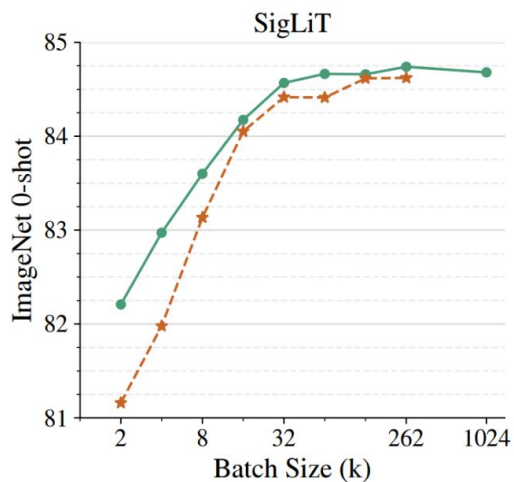
from row-col two-step softmax application to the pairwise sigmoid normalization(?). may add bias b as sigmoid wouldn't cancel it. $z_{\{ij\}}$ is the mask for the diag..

Consider bias b as the handicap for the pos. pair instances in the diag. due to the somewhat high influence of class imbalance in the earlier stage (1:16k, 1.6k, .)

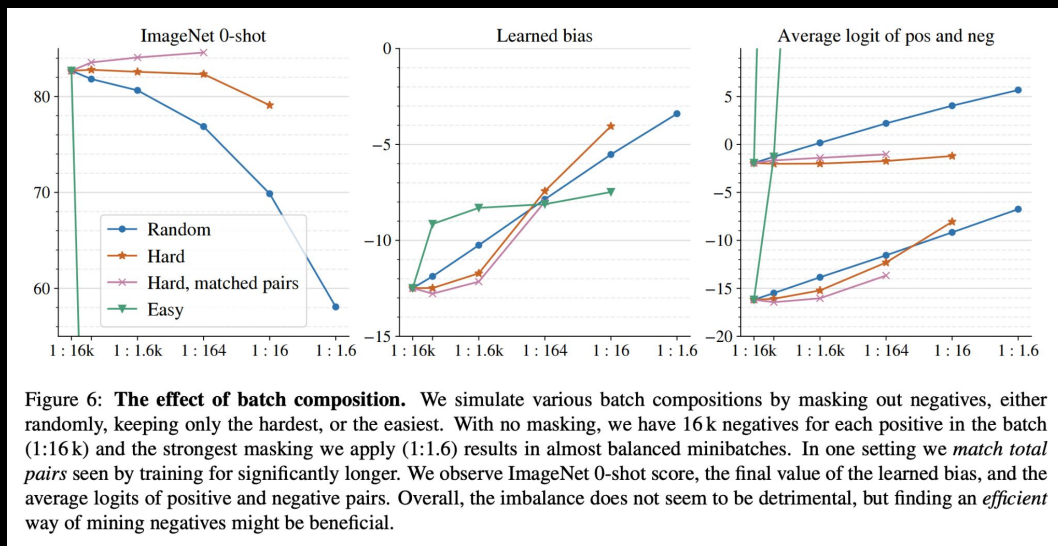
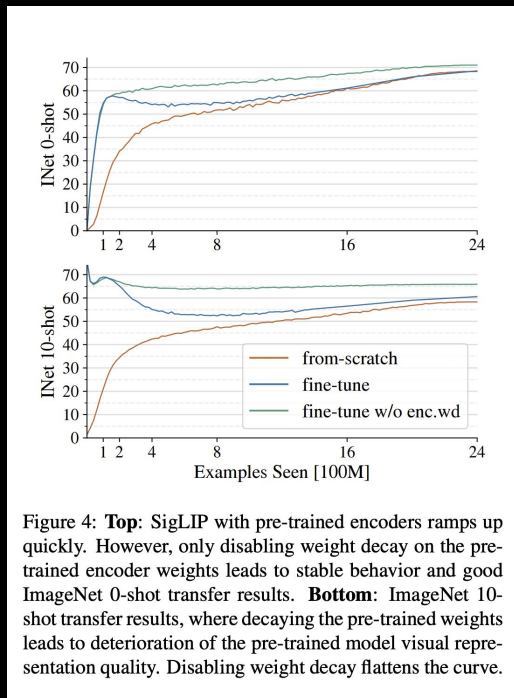


$$-\frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \sum_{j=1}^{|\mathcal{B}|} \underbrace{\log \frac{1}{1 + e^{z_{ij}(-t\mathbf{x}_i \cdot \mathbf{y}_j + b)}}}_{\mathcal{L}_{ij}}$$

SigLip in a nutshell



SigLip in a nutshell



Use hard problems (match between dog, retriever, etc)